

SAPIENZA
UNIVERSITÀ DI ROMA

PageRank: Markov Chains and GPU Computing

Faculty of Information Engineering,
Informatics and Statistics
Bachelor's Degree in Computer and Control Engineering

Marco Mazzocco
ID number 2136444

Advisor
Federico Fusco

Academic Year 2025/2026

PageRank: Markov Chains and GPU Computing
Final technical report. Sapienza University of Rome

© 2026 Marco Mazzocco. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Website: https://mazzocco51.github.io
Author's email: mazzocco.dev@gmail.com

Abstract

PageRank is an algorithm that measures the importance of nodes in a directed graph through its link structure. This thesis studies PageRank as the stationary distribution of a Markov chain and its computation on GPUs. After its existence and uniqueness are examined, power iteration and the Monte Carlo method are implemented and evaluated on real graphs ranging from approximately 81 thousand to 62 million nodes. On *web-Google*, GPU power iteration achieves a speedup of 16.6× over the baseline CPU version, reaching 37.1× with suitable optimisations. On *GAP-twitter*, a CUDA kernel reduces the memory occupied by the graph matrix from approximately 12 to 6 GB. Finally, a contact recommendation system for social networks is developed using Personalized PageRank: for 506 of the 5,000 users evaluated, at least one suggested profile appears among the new contacts observed in the following month.

Contents

1 Introduction **1**

2 Markov Chains **2**

2.1 Stochastic Processes and the Markov Property 2
2.2 Transition Probability Matrix 2
2.3 Evolution and m-Step Transitions 3
2.4 Graph Representation 3
2.5 Accessibility and Irreducibility 5
2.6 Recurrence, Periodicity and Ergodicity 5
2.7 Stationary Distribution and Convergence 6
2.8 Perron's Theorem 7

3 PageRank **10**

3.1 Random Surfer Model 10
3.2 Flow Equations 10
3.3 Probabilistic Interpretation 13
3.4 Limitations of the Model 13
3.5 The Google Matrix 14
3.6 Existence and Uniqueness of PageRank 15
3.7 Rate of Convergence 15
3.8 Choosing the Damping Factor 17

4 Computational Algorithms **18**

4.1 Power Iteration 18
4.2 Sparse Computation 19
4.3 Stopping Criterion 22
4.4 Monte Carlo Method 23
4.5 Comparison 24

5 Implementation **25**

5.1 Execution Environment 25
5.2 Graphs and Data Structures 25
5.3 Power Iteration on CPUs and GPUs 27
5.3.1 Execution Times 27
5.3.2 Convergence for Different Values of α 28
5.4 Monte Carlo on GPUs 30
5.5 Large-Scale PageRank 30
5.5.1 Data Preparation 31
5.5.2 Power Iteration 31
5.5.3 Monte Carlo Method 31
5.5.4 Reducing Memory with a CUDA Kernel 31
5.6 Contact Recommendation 33
5.6.1 Personalized PageRank 33
5.6.2 Experimental Protocol 34
5.6.3 Results 34
5.7 Conclusions 36

Bibliography **37**

Chapter 1

Introduction

Searching for information on the Web requires identifying the pages relevant to a query and ranking them. These tasks belong to the field of *information retrieval* [EK]. This is the setting for *PageRank*, an algorithm introduced in 1998 by Sergey Brin and Larry Page [BP], the co-founders of Google.

The algorithm assesses the importance of Web pages through the structure of the links connecting them. The idea is that a link to a page contributes to its importance, especially if it comes from a page that is itself important. Brin and Page consider a user who browses by randomly following links between pages and occasionally becomes bored and starts again from a randomly chosen page [BP]. A page's PageRank is the probability of finding the surfer there in the long run. In the model, the choice of the next page depends only on the current one: the successive moves define a *Markov chain*. We must therefore establish when PageRank is well defined and how to compute it for networks with millions of pages.

The network of links can be represented as a directed graph, whose nodes are the pages and whose edges are the links. PageRank therefore also applies to other graphs of this kind, such as social networks. The computational methods repeat the same simple operation on many independent elements, which makes them suitable for parallel execution on a GPU.

This thesis studies PageRank as the stationary distribution of a Markov chain, establishing the conditions for its existence and uniqueness and the rate at which the chain converges to it. Two methods for computing PageRank, power iteration and the Monte Carlo method, are presented, and their cost and accuracy are analysed. Both methods are implemented on a GPU and tested on real graphs of increasing size. For a large-scale graph, a CUDA kernel is developed to reduce the memory required. Finally, power iteration is adapted to Personalized PageRank to build and evaluate a contact recommendation system for a social network.

The thesis is organised as follows. Chapter 2 introduces Markov chains, the stationary distribution and the fundamental theorems. Chapter 3 presents the PageRank model and studies its existence, uniqueness and rate of convergence. Chapter 4 describes the computational methods, with a stopping criterion and an error estimate. Chapter 5 presents the CPU and GPU implementations, the experiments, the application to contact recommendation and the conclusions.

Chapter 2

Markov Chains

In the random-surfer model underlying PageRank, the page visited at step $t+1$ depends only on the page at step t, regardless of the sequence of previous visits: the dynamics satisfy the Markov, or memoryless, property.

We introduce the concepts from the theory of stochastic processes needed to formalise this model and study its asymptotic behaviour. Since the Web consists of a finite set of pages, we restrict our attention to Markov chains on finite state spaces.

2.1 Stochastic Processes and the Markov Property

A discrete-time stochastic process is a sequence of random variables $X = (X_t)_{t\in\mathbb{N}}$ taking values in a finite set S, called the *state space*. Without loss of generality, we assume $S = \{0, 1, \ldots, n-1\}$.

Definition 2.1 (*Markov property*)**.** A stochastic process X satisfies the Markov property if, for every time $t \in \mathbb{N}$ and every sequence $i_0, \ldots, i_{t-1}, i, j \in S$,

$$\Pr(X_{t+1} = j \mid X_t = i,\, X_{t-1} = i_{t-1},\, \ldots,\, X_0 = i_0) = \Pr(X_{t+1} = j \mid X_t = i).$$

A process satisfying this property is called a *Markov chain*. The quantity $\Pr(X_{t+1} = j \mid X_t = i)$ is the *transition probability* from state i to state j. If it does not depend on t, it is denoted by $P_{i,j}$, and the chain is said to be *time-homogeneous*. Only chains of this kind will be considered below.

2.2 Transition Probability Matrix

The dynamics of a homogeneous Markov chain are described by a *one-step transition probability matrix*:

$$\mathbf{P} = \begin{pmatrix} P_{0,0} & P_{0,1} & \cdots & P_{0,n-1} \\ P_{1,0} & P_{1,1} & \cdots & P_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n-1,0} & P_{n-1,1} & \cdots & P_{n-1,n-1} \end{pmatrix}$$

The entry in row i and column j is the transition probability $P_{i,j}$. Since at every step the chain necessarily moves to one of the states in S, we have:

$$\sum_{j \in S} P_{i,j} = 1 \qquad \text{for all } i \in S.$$

An $n \times n$ matrix whose entries are all nonnegative and whose rows sum to 1 is called a *stochastic matrix*.

2.3 Evolution and m-Step Transitions

Let $p_i(t) = \Pr(X_t = i)$ be the probability that the process is in state i at time t; note that, unlike $P_{i,j}$, $p_i(t)$ is unconditional and generally changes at each step. Also, let

$$\bar{p}(t) = (p_0(t), p_1(t), \ldots, p_{n-1}(t))$$

be the distribution of X_t. Summing over all possible states at time t gives:

$$p_i(t+1) = \sum_{j \in S} p_j(t) P_{j,i}$$

or, in matrix form,

$$\bar{p}(t+1) = \bar{p}(t)\mathbf{P}. \tag{2.1}$$

The distribution is written as a *row vector* so that the notation follows the process: applying the operator $\mathbf{P}$ on the right yields the distribution at the next step. For every $m \geq 0$, the *m-step transition probability* is defined as

$$P_{i,j}^{(m)} = \Pr(X_{t+m} = j \mid X_t = i).$$

Let $\mathbf{P}^{(m)}$ be the *m-step transition probability matrix*. For $m \geq 1$, isolating the first step gives

$$\mathbf{P}^{(m)} = \mathbf{P}\,\mathbf{P}^{(m-1)},$$

and, since $\mathbf{P}^{(0)} = \mathbf{I}$, induction on m yields

$$\mathbf{P}^{(m)} = \mathbf{P}^m.$$

Thus

$$\bar{p}(t+m) = \bar{p}(t)\,\mathbf{P}^m \qquad \text{for all } t \in \mathbb{N},\ m \geq 0.$$

2.4 Graph Representation

A Markov chain can be represented by a weighted directed graph $D = (V, E, w)$. Its vertex set coincides with the state space of the chain, while its edge set is defined as

$$E = \{(i,j) \in V \times V : P_{i,j} > 0\},$$

and edge (i,j) has weight $w_{i,j} = P_{i,j}$. Stochasticity becomes the condition

$$\sum_{j\,:\,(i,j) \in E} w_{i,j} = 1 \qquad \text{for all } i \in V.$$

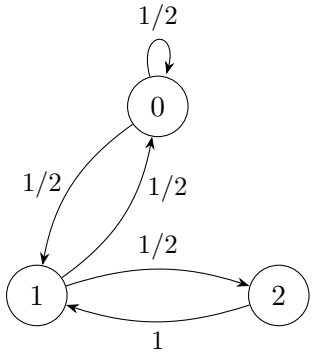
A sequence of states $i_0, i_1, \ldots, i_m$ corresponds to a *directed path in the graph* D. By the Markov property, the probability of following this path starting from i_0 is the product of the weights of its edges:

$$\Pr(X_1 = i_1, X_2 = i_2, \ldots, X_m = i_m \mid X_0 = i_0) = \prod_{k=0}^{m-1} w_{i_k, i_{k+1}}.$$

$P_{i,j}^{(m)}$ is the sum, over all paths of length m from i to j, of the products of their edge weights. Since the weights are positive, $P_{i,j}^{(m)} > 0$ if and only if at least one such path exists in D.

Example 2.2. Consider a Markov chain with state space $S = \{0, 1, 2\}$:

$$\mathbf{P} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \end{pmatrix} \qquad \longleftrightarrow$$



In particular, $P_{1,2} = w_{1,2} = 1/2$. The weights of the edges leaving state 1 satisfy

$$\sum_{j:\,(1,j)\in E} w_{1,j} = w_{1,0} + w_{1,2} = \frac{1}{2} + \frac{1}{2} = 1.$$

In the graph, the path $0 \to 1 \to 2 \to 1$ has probability

$$\Pr(X_1 = 1, X_2 = 2, X_3 = 1 \mid X_0 = 0) = w_{0,1} \cdot w_{1,2} \cdot w_{2,1} = \frac{1}{2} \cdot \frac{1}{2} \cdot 1 = \frac{1}{4}. \qquad \triangleleft$$

2.5 Accessibility and Irreducibility

Definition 2.3 (*Accessibility and communication*)**.** State j is *accessible* from state i if there is an integer $m \geq 0$ such that $P_{i,j}^{(m)} > 0$, or, equivalently, if a path from i to j exists in D. If two states are accessible from each other, they *communicate*, written $i \leftrightarrow j$ [MU, Ch. 7].

Communication is an equivalence relation: symmetry is inherent in the definition, transitivity follows from concatenating paths, and reflexivity holds because $\mathbf{P}^0 = \mathbf{I}$, so every state is accessible from itself with $m = 0$. The relation therefore partitions the states into equivalence classes, called *communicating classes*.

Definition 2.4 (*Irreducibility*)**.** A Markov chain is *irreducible* if all its states belong to a single communicating class; otherwise it is *reducible*.

Lemma 2.5. *A Markov chain is irreducible if and only if its associated graph D is strongly connected.*

Proof. By Definition 2.4, a chain is irreducible if its states form a single communicating class, that is, if for every ordered pair of states (i, j) state j is accessible from i. As shown in Section 2.4, j is accessible from i if and only if D contains a directed path from i to j. The existence of a directed path for every ordered pair of vertices is precisely the definition of a strongly connected graph. □

2.6 Recurrence, Periodicity and Ergodicity

The asymptotic behaviour of a Markov chain is determined by two independent properties of its states: whether a return to a state is certain (*recurrence*), and the temporal regularity with which it occurs (*periodicity*).

Definition 2.6 (*Recurrence and transience*)**.** A state i is *recurrent* if, starting from i, the chain returns to it with probability 1; otherwise it is *transient* [MU, Ch. 7].

Every visit to a recurrent state implies another: this happens infinitely often with probability 1, by the memoryless property (Definition 2.1), so the sequence of returns is infinite. By contrast, the total number of visits to a transient state is finite with probability 1. Recurrence and transience are class properties: within a communicating class, either all states are recurrent or all are transient.

Lemma 2.7. *A finite-state Markov chain always has at least one recurrent state.*

Proof. We proceed by contradiction. If all states were transient, each would be visited finitely many times with probability 1. However, the process is defined for every $t \in \mathbb{N}$, and the chain visits a state at each time: infinitely many times and finitely many states imply that at least one state is visited infinitely often. A state visited infinitely often cannot be transient, which is a contradiction. □

A recurrent class is *closed*: if it were possible to leave it, returning would be impossible, since otherwise the outside states traversed would belong to the same communicating class. Leaving permanently would violate recurrence. When the class contains only some of the states, the process cannot reach the others once it has entered the class.

Definition 2.8 (*Periodicity*)**.** A state i is *periodic* if there is an integer $k > 1$ such that a return to i is possible only after a number of steps divisible by k; otherwise it is *aperiodic*. A chain is periodic if at least one of its states is periodic.

Within a communicating class, either all states are periodic or all are aperiodic. An irreducible chain is aperiodic if at least one of its states is aperiodic.

Definition 2.9 (*Ergodicity*)**.** In a finite Markov chain, a state is called *ergodic* if it is recurrent and aperiodic. The chain is called *ergodic* if all its states are ergodic [MU, Def. 7.7 and Lemma 7.5].

Corollary 2.10. *Every finite, irreducible and aperiodic Markov chain is ergodic [MU, Cor. 7.6].*

2.7 Stationary Distribution and Convergence

If $\mathbf{P}$ is the one-step transition matrix of a Markov chain and $\bar{p}(t)$ is the probability distribution of its state at time t, then

$$\bar{p}(t+1) = \bar{p}(t)\mathbf{P}.$$

Definition 2.11 (*Stationary distribution*)**.** A *stationary distribution* of a Markov chain is a probability distribution $\bar{\pi}$ such that

$$\bar{\pi} = \bar{\pi}\mathbf{P}.$$

Once a chain reaches this distribution, it retains it for all future times.

Definition 2.12 (*Mean return time*)**.** The *mean return time* $h_{i,i}$ is the expected number of steps that a chain starting at i takes to return to i. In a finite irreducible chain, every state is recurrent and has a finite mean return time:

$$h_{i,i} = \mathbb{E}[\min\{t \geq 1 : X_t = i\} \mid X_0 = i].$$

Theorem 2.13 (Fundamental theorem of Markov chains [MU, Thm. 7.7])**.** *Every finite, irreducible and ergodic Markov chain has the following properties:*

1. *the chain has a unique stationary distribution* $\bar{\pi} = (\pi_0, \pi_1, \pi_2, \ldots, \pi_{n-1})$*;*
2. *for any pair of states* (i, j)*, the limit* $\lim_{t\to\infty} P_{i,j}^{(t)}$ *exists and is independent of* i*;*
3. $\pi_j = \lim_{t\to\infty} P_{i,j}^{(t)} = \frac{1}{h_{j,j}}$.

The component π_j is the reciprocal of the mean return time to j. For an irreducible chain, the stationary distribution is unique; if the chain is also aperiodic, the probability of being in j converges to π_j, regardless of the initial distribution.

The stationary distribution can be computed by solving the system

$$\bar{\pi} = \bar{\pi}\mathbf{P}, \qquad \sum_{i\in S} \pi_i = 1.$$

In the irreducible case considered here, the normalisation selects the unique stationary distribution.

2.8 Perron's Theorem

The equation $\bar{\pi}\mathbf{P} = \bar{\pi}$ expresses the fact that the stationary distribution remains unchanged after a transition. Thus $\bar{\pi}$ is a *left eigenvector* of $\mathbf{P}$ corresponding to the *eigenvalue* 1. This eigenvalue formulation underlies the computation of PageRank.

Lemma 2.14. *Let* $\mathbf{P}$ *be a row-stochastic matrix. Then* 1 *is an eigenvalue of* $\mathbf{P}$ *and every eigenvalue* λ *satisfies* $|\lambda| \leq 1$.

Proof. Let $\mathbf{e}$ be the column vector whose entries are all equal to 1. Since each row of $\mathbf{P}$ sums to 1, we have $\mathbf{Pe} = \mathbf{e}$; hence 1 is an eigenvalue.

Consider a left eigenvector $\bar{x} \neq \bar{0}$ of $\mathbf{P}$ associated with any eigenvalue λ (for $n \times n$ matrices, left and right eigenvalues coincide). The equality $\bar{x}\mathbf{P} = \lambda\bar{x}$ gives, for every component j,

$$\sum_i x_i P_{ij} = \lambda x_j.$$

Taking absolute values and summing over j, we obtain

$$\begin{aligned}
|\lambda| \sum_j |x_j| &= \sum_j \left| \sum_i x_i P_{ij} \right| \\
&\leq \sum_j \sum_i |x_i| P_{ij} \qquad \text{(triangle inequality)} \\
&= \sum_i |x_i| \sum_j P_{ij} \\
&= \sum_i |x_i|. \qquad \text{(stochasticity)}
\end{aligned}$$

Since $\bar{x} \neq \bar{0}$, we have $\sum_i |x_i| > 0$. Dividing, we conclude that $|\lambda| \leq 1$. □

For matrices with strictly positive entries, the following result holds [LMB, Sec. 15.2].

Theorem 2.15 (Perron's theorem)**.** *Let* $\mathbf{A}$ *be an* $n \times n$ *matrix with strictly positive real entries. Then:*

1. *there is a real eigenvalue* $r > 0$ *such that* $r > |\lambda|$ *for every other eigenvalue* λ *of* $\mathbf{A}$*;*
2. *associated with* r *is a left eigenvector* $\bar{y}$ *with positive real entries, unique up to multiplication by a constant;*
3. *if* $r = 1$*, then, for any nonzero initial vector* $\bar{x}$ *with nonnegative entries, the sequence* $\bar{x}\mathbf{A}^k$ *converges, as* $k \to \infty$*, to a vector in the direction of* $\bar{y}$*.*

By Lemma 2.14, for a stochastic matrix $\mathbf{P}$, the eigenvalue 1 has maximum modulus. If the entries of $\mathbf{P}$ are strictly positive, then $r = 1$ and

$$\bar{y}\mathbf{P} = r\,\bar{y} = \bar{y}.$$

Normalising $\bar{y}$ gives

$$\bar{\pi} = \frac{\bar{y}}{\sum_{i \in S} y_i}, \qquad \bar{\pi}\mathbf{P} = \frac{\bar{y}\mathbf{P}}{\sum_{i \in S} y_i} = \bar{\pi}.$$

By the third part of Theorem 2.15, for any initial distribution $\bar{p}(0)$, $\bar{p}(0)\mathbf{P}^k$ converges to a positive multiple of $\bar{y}$. However, the entries of every distribution obtained during the evolution sum to 1, and the limit must also sum to 1. This multiple can therefore only be $\bar{\pi}$.
Since $P_{i,j} > 0$ for every pair of states (i, j),

- any j can be reached from any i in one step: the chain is irreducible;
- a one-step return is possible from any i, since $P_{i,i} > 0$: the chain is aperiodic.

This recovers the convergence result of Theorem 2.13 by an algebraic argument.

The fundamental theorem guarantees that the stationary distribution exists, is unique and is reached as a limit. Neither theorem, however, quantifies the rate of convergence, which depends on the spectral structure of $\mathbf{P}$.

Let $\mathbf{P}$ be the transition matrix of a finite, irreducible and aperiodic chain. By Corollary 2.10 and Theorem 2.13, the powers $\mathbf{P}^k$ converge to the rank-one matrix $\mathbf{e}\bar{\pi}$. This implies that the eigenvalue 1 is simple and that all other eigenvalues have modulus strictly less than 1. The eigenvalues can therefore be ordered so that

$$1 = \lambda_1 > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|.$$

Since $\bar{\pi}\mathbf{P}^k = \bar{\pi}$, we have $(\bar{p}(0) - \bar{\pi})\mathbf{P}^k = \bar{p}(0)\mathbf{P}^k - \bar{\pi}$.
Convergence to $\bar{\pi}$ therefore corresponds to the decay to zero of the deviation $\bar{p}(0)\mathbf{P}^k - \bar{\pi}$.

For a left eigenvector $\bar{y}_i$ associated with the eigenvalue λ_i, we have $\bar{y}_i\mathbf{P}^k = \lambda_i^k \bar{y}_i$. When the matrix is diagonalisable, the initial deviation $\bar{p}(0) - \bar{\pi}$ can be written as a combination of eigenvectors. The stationary component is zero, since $\bar{p}(0)$ and $\bar{\pi}$ both sum to one. Each application of $\mathbf{P}$ multiplies the eigenvector $\bar{y}_i$ by λ_i. Since $|\lambda_i| < 1$ for $i \geq 2$, the deviation tends to zero.

Among the eigenvalues other than 1, λ_2 has the largest modulus, so the factor $|\lambda_2|^k$ is the one that decays most slowly. The *spectral gap* is defined as

$$\gamma = 1 - |\lambda_2|.$$

A larger spectral gap indicates faster asymptotic convergence to $\bar{\pi}$.

Chapter 3

PageRank

In the initial PageRank model, each page is assigned a score that depends on its incoming links and on the importance of the pages they come from. At each step, every page:

- distributes its score equally through its outgoing links (*outlinks*);
- receives contributions from the pages that cite it, through its incoming links (*backlinks*).

The sum of the contributions received becomes its new score.

We first introduce the link-based model, and then address the problems caused by the structure of the Web graph, such as dangling nodes and closed classes.

3.1 Random Surfer Model

A Web page is considered important if it is visited frequently. Since traffic data are not available on a global scale, a user is modelled as a *random surfer* who follows links at random. The aim is to define the PageRank of a page i as the limiting probability that the surfer is at i.

The model assumes that the surfer chooses one of the outgoing links of the current page with equal probability, independently of the pages visited previously. Browsing therefore satisfies the Markov property (Definition 2.1) and can be described by a transition matrix.

3.2 Flow Equations

The Web is represented by a directed graph $D = (V, E)$, where:

- $V = \{0, \dots, n-1\}$ is the set of pages;
- $E = \{(i, j) \in V \times V : \text{there is a link from page } i \text{ to page } j\}$ is the set of links.

The out-degree d_i^+ is the number of outlinks of page i:

$$d_i^+ = \big|\{j \in V : (i, j) \in E\}\big|.$$

We initially assume that $d_i^+ > 0$ for every $i \in V$.

Let $r_i(t)$ denote the score of page i at step t. Each page distributes its score equally among its outlinks, contributing $r_i(t)/d_i^+$ to each destination. The score *update rule* is

$$r_j(t+1) = \sum_{i:\, i \to j} \frac{r_i(t)}{d_i^+}. \tag{3.1}$$

At equilibrium, the scores do not change from one step to the next:

$$r_j = \sum_{i:\, i \to j} \frac{r_i}{d_i^+}. \tag{3.2}$$

The flow equations can be written compactly using the matrix $\mathbf{P}$, where

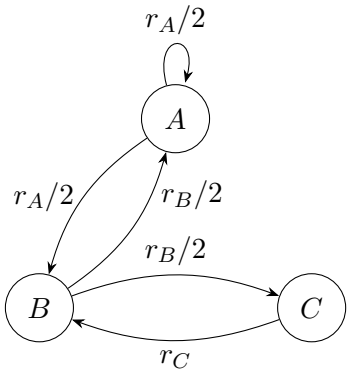
$$P_{ij} = \begin{cases} 1/d_i^+, & \text{if } (i,j) \in E, \\ 0, & \text{otherwise.} \end{cases} \tag{3.3}$$

Equations (3.1) and (3.2) become

$$\bar{r}(t+1) = \bar{r}(t)\mathbf{P}, \qquad \bar{r} = \bar{r}\mathbf{P}.$$

The ith row of $\mathbf{P}$ distributes the score of i among its outlinks.
Under the assumption $d_i^+ > 0$, each row of $\mathbf{P}$ sums to 1: $\mathbf{P}$ is stochastic, and a nonnegative normalised score vector $\bar{r}$ satisfying $\bar{r} = \bar{r}\mathbf{P}$ is a stationary distribution in the sense of Definition 2.11.

Figure 3.1. Graph of three pages A, B and C. Each edge shows the score transmitted, r_i/d_i^+.



Example 3.1. Consider the graph in Figure 3.1. Pages A and B each have two outlinks, while C links only to B.

The flow equations therefore become

$$\begin{cases} r_A = \dfrac{r_A}{2} + \dfrac{r_B}{2}, \\ r_B = \dfrac{r_A}{2} + r_C, \\ r_C = \dfrac{r_B}{2}. \end{cases}$$

The system is homogeneous and determines the ratios between the scores, rather than their values: the equations give $r_A = r_B = 2r_C$, and imposing

$$\sum_{j \in V} r_j = 1$$

gives

$$r_A = r_B = \frac{2}{5}, \qquad r_C = \frac{1}{5}.$$

By (3.3), the matrix of the graph is

$$\mathbf{P} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \end{pmatrix}.$$

The resulting vector satisfies

$$\left(\frac{2}{5}, \frac{2}{5}, \frac{1}{5}\right) \mathbf{P} = \left(\frac{2}{5}, \frac{2}{5}, \frac{1}{5}\right). \qquad \triangleleft$$

3.3 Probabilistic Interpretation

The score $r_i(t)$ of page i can be interpreted as the probability $p_i(t)$ that the random surfer is on that page after t steps. The surfer chooses the initial page uniformly at random among the n available pages, each with probability $1/n$. Each page is therefore assigned the initial score $r_i(0) = 1/n$. Writing $\bar{p}(t)$ for the distribution of the surfer's position, we have

$$\bar{r}(0) = \bar{p}(0) = \left(\tfrac{1}{n}, \ldots, \tfrac{1}{n}\right).$$

From page i, the surfer chooses each outlink with probability $1/d_i^+$. The probability of moving from i to j therefore equals the entry P_{ij} defined in (3.3). The surfer satisfies the Markov property and thus has $\mathbf{P}$ as its transition matrix. By (2.1),

$$\bar{p}(t+1) = \bar{p}(t)\mathbf{P}.$$

Since the two vectors $\bar{p}(t)$ and $\bar{r}(t)$ are initially equal and follow the same update rule, they are equal at every step.

3.4 Limitations of the Model

The model outlined above assumes that each page has at least one outgoing link, that is, $d_i^+ > 0$. The actual structure of the Web, however, systematically violates this assumption. Pages with no outgoing links are called *dangling nodes*: when the surfer visits one, there is no link with which to continue browsing (node D in Figure 3.2). A dangling node i satisfies $d_i^+ = 0$ and, by (3.3),

$$P_{ij} = 0 \text{ for all } j \in V.$$

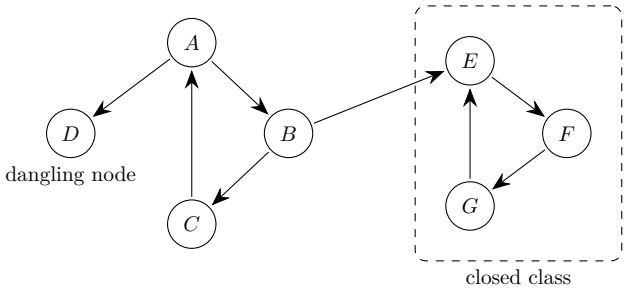
The ith row of $\mathbf{P}$ is then entirely zero, and $\mathbf{P}$ is no longer stochastic. The node's score is not distributed at the next step, so the total score may decrease. $\mathbf{P}$ is not the transition matrix of any Markov chain, and the results of Chapter 2 do not apply.

If a subset of Web pages communicate (Definition 2.3) only with one another, with no outgoing links to other pages, this subset forms a *closed communicating class* (nodes E, F, G in Figure 3.2). Once the surfer enters the class, it remains there forever.

In the absence of dangling nodes, if the network has *a single* proper closed class, the stationary distribution $\bar{r}$ exists and is unique, but it is concentrated in that class; pages outside it receive a score of zero, regardless of their number of incoming links. If *two or more* closed classes coexist, equation (3.2) admits infinitely many stationary distributions. When the iteration has a limit, this limit may depend on the initial distribution: different initialisations may produce different scores for the same page.

A graph containing a proper closed class is not strongly connected. Hence the associated Markov chain is not irreducible (Lemma 2.5), and the fundamental theorem (Theorem 2.13) does not apply.

Figure 3.2. Node D is dangling ($d_D^+ = 0$); nodes E, F, G form a closed communicating class.



The link structure of the Web does not guarantee aperiodicity either. In the presence of periodic states (Definition 2.8), the distribution of the surfer's position may oscillate without converging, even when the stationary distribution is unique (the cycle $E \to F \to G \to E$ in Figure 3.2).

3.5 The Google Matrix

The three limitations of the model are overcome by a single idea: allowing the surfer to reach any page even without following a link. When the surfer reaches a page with no outlinks, it is assumed to choose any of the n Web pages with equal probability. Each zero row of $\mathbf{P}$ is therefore replaced by a row whose entries all equal $1/n$. This yields the matrix $\mathbf{S}$, defined by

$$S_{ij} = \begin{cases} P_{ij}, & \text{if } d_i^+ > 0, \\ 1/n, & \text{if } d_i^+ = 0. \end{cases}$$

Each row of $\mathbf{S}$ sums to 1: $\mathbf{S}$ is a stochastic matrix.

To deal with closed classes and periodic behaviour, *teleportation* is introduced: at each step, with probability α the surfer follows the transitions described by $\mathbf{S}$, while with probability $1 - \alpha$ it chooses any of the n pages uniformly at random, with $0 < \alpha < 1$.

The resulting matrix, called the *Google matrix*, has entries

$$G_{ij} = \alpha S_{ij} + (1 - \alpha)\frac{1}{n}. \tag{3.4}$$

The first term describes the transitions according to $\mathbf{S}$, and the second describes teleportation. α is called the *damping factor*. Equivalently,

$$\mathbf{G} = \alpha \mathbf{S} + \frac{1 - \alpha}{n}\, \mathbf{e}\mathbf{e}^T, \tag{3.5}$$

where $\mathbf{e}$ is the column vector whose components all equal 1.

The matrix $\mathbf{G}$ is stochastic: each row of $\mathbf{S}$ sums to 1, and there are n teleportation terms, each equal to $(1 - \alpha)/n$, so $\sum_{j \in V} G_{ij} = \alpha + (1 - \alpha) = 1$. All its entries are also strictly positive, since $(1 - \alpha)/n > 0$.

3.6 Existence and Uniqueness of PageRank

The matrix $\mathbf{G}$ is strictly positive, so it satisfies the assumption of Theorem 2.15. The same property also establishes the assumptions of Theorem 2.13:

1. for every pair of pages (i, j), $G_{i,j} > 0$, so the associated graph is strongly connected and the chain corresponding to $\mathbf{G}$ is irreducible;
2. in particular, $G_{i,i} > 0$: a return in one step is possible from every state i, and the chain is aperiodic.

The chain is finite, irreducible and aperiodic, hence ergodic by Corollary 2.10. Theorem 2.13 guarantees a unique stationary distribution $\bar{\pi}$ and the convergence

$$\lim_{t\to\infty} \bar{p}(0)\mathbf{G}^t = \bar{\pi}, \tag{3.6}$$

for any initial distribution $\bar{p}(0)$.

Definition 3.2 (*PageRank vector*)**.** The PageRank vector, denoted by $\bar{r}$, is the stationary distribution of the Google matrix $\mathbf{G}$. Its component r_i is the PageRank score of page i.

Theorem 2.13 also gives $r_i = \frac{1}{h_{i,i}}$, where $h_{i,i}$ is the mean return time to page i in the chain associated with $\mathbf{G}$. A higher score corresponds to a shorter mean return time (that is, the random surfer visits page i more frequently).

The PageRank vector satisfies

$$\bar{r}\mathbf{G} = \bar{r}. \tag{3.7}$$

It is therefore a left eigenvector of $\mathbf{G}$ associated with the eigenvalue 1.

By Lemma 2.14, 1 is an eigenvalue of $\mathbf{G}$ with maximum modulus. Finally, Theorem 2.15 ensures that every other eigenvalue λ has modulus $|\lambda| < 1$ and that the left eigenvector associated with 1 is unique up to a multiplicative factor.

3.7 Rate of Convergence

The relationship between the rate of convergence and the spectrum of the matrix, discussed in Section 2.8, reduces the problem to the study of the eigenvalue of $\mathbf{G}$ with the second-largest modulus.

Theorem 3.3 ([LM, Thm. 5.1, pp. 346–347])**.** *Let* $\mathbf{S}$ *be a stochastic matrix with spectrum* $\{1, \lambda_2, \lambda_3, \ldots, \lambda_n\}$*. Then the matrix* $\mathbf{G}$ *defined in* (3.5)*,*

$$\mathbf{G} = \alpha\mathbf{S} + \frac{1-\alpha}{n}\,\mathbf{e}\mathbf{e}^T,$$

has spectrum $\{1, \alpha\lambda_2, \alpha\lambda_3, \ldots, \alpha\lambda_n\}$*.*

Proof. We choose a basis in which $\mathbf{S}$ and $\mathbf{G}$ take block triangular form. The resulting matrices are similar to the original ones and therefore share their spectra. Since $\mathbf{S}$ is

row-stochastic, $\mathbf{Se} = \mathbf{e}$. We extend $\mathbf{e}$ to a basis of $\mathbb{R}^n$ and write the change-of-basis matrix and its inverse as

$$\mathbf{Q} = \begin{pmatrix} \mathbf{e} & \mathbf{X} \end{pmatrix}, \qquad \mathbf{Q}^{-1} = \begin{pmatrix} \bar{u} \\ \mathbf{Y} \end{pmatrix}.$$

The identity $\mathbf{Q}^{-1}\mathbf{Q} = \mathbf{I}_n$ gives

$$\begin{pmatrix} \bar{u}\mathbf{e} & \bar{u}\mathbf{X} \\ \mathbf{Ye} & \mathbf{YX} \end{pmatrix} = \begin{pmatrix} 1 & \mathbf{0}^T \\ \mathbf{0} & \mathbf{I}_{n-1} \end{pmatrix},$$

hence $\bar{u}\mathbf{e} = 1$ and $\mathbf{Ye} = \mathbf{0}$. Using these identities and $\mathbf{Se} = \mathbf{e}$, we obtain

$$\mathbf{Q}^{-1}\mathbf{SQ} = \begin{pmatrix} \bar{u} \\ \mathbf{Y} \end{pmatrix} \begin{pmatrix} \mathbf{e} & \mathbf{SX} \end{pmatrix} = \begin{pmatrix} 1 & \bar{u}\mathbf{SX} \\ \mathbf{0} & \mathbf{B} \end{pmatrix},$$

where $\mathbf{B} = \mathbf{YSX}$. The matrix is block triangular, so

$$\det(\mathbf{Q}^{-1}\mathbf{SQ} - \lambda\mathbf{I}_n) = (1 - \lambda)\det(\mathbf{B} - \lambda\mathbf{I}_{n-1}).$$

Since similarity preserves the characteristic polynomial, the eigenvalues of $\mathbf{B}$ are $\lambda_2, \ldots, \lambda_n$, counted with their algebraic multiplicities.

For the teleportation term, note that

$$\mathbf{Q}^{-1}\mathbf{e} = \begin{pmatrix} 1 \\ \mathbf{0} \end{pmatrix}, \qquad \mathbf{e}^T\mathbf{Q} = \begin{pmatrix} n & \mathbf{e}^T\mathbf{X} \end{pmatrix},$$

where the identity $\mathbf{e}^T\mathbf{e} = n$ was used. Thus

$$\mathbf{Q}^{-1}\mathbf{e}\mathbf{e}^T\mathbf{Q} = \begin{pmatrix} 1 \\ \mathbf{0} \end{pmatrix} \begin{pmatrix} n & \mathbf{e}^T\mathbf{X} \end{pmatrix} = \begin{pmatrix} n & \mathbf{e}^T\mathbf{X} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}.$$

Substituting into the definition of $\mathbf{G}$,

$$\mathbf{Q}^{-1}\mathbf{GQ} = \alpha(\mathbf{Q}^{-1}\mathbf{SQ}) + \frac{1-\alpha}{n}(\mathbf{Q}^{-1}\mathbf{e}\mathbf{e}^T\mathbf{Q}) = \begin{pmatrix} 1 & * \\ \mathbf{0} & \alpha\mathbf{B} \end{pmatrix},$$

where the first diagonal block equals $\alpha + (1 - \alpha) = 1$ and $*$ denotes a block that does not affect the spectrum. The eigenvalues of the resulting block triangular matrix are 1 and those of $\alpha\mathbf{B}$, namely $\alpha\lambda_2, \ldots, \alpha\lambda_n$. By similarity, $\mathbf{G}$ has the same spectrum. □

By Lemma 2.14, the eigenvalues of $\mathbf{S}$ have modulus at most 1. Theorem 3.3 shows that teleportation preserves one eigenvalue equal to 1 and multiplies all the others by α. Therefore,

$$|\lambda_2(\mathbf{G})| = \alpha|\lambda_2(\mathbf{S})| \leq \alpha.$$

Teleportation thus separates the eigenvalue 1 from all the others in modulus, even when $\mathbf{S}$ has several eigenvalues of modulus 1.

Example 3.4. Suppose that $\mathbf{S}$ has eigenvalues $1,\ 1,\ -1,\ 0.6$. With $\alpha = 0.85$, Theorem 3.3 gives the eigenvalues of $\mathbf{G}$:

$$1,\ 1,\ -1,\ 0.6 \quad \longrightarrow \quad 1,\ 0.85,\ -0.85,\ 0.51.$$

The non-dominant eigenvalues therefore have modulus at most 0.85, and their distance from 1 is at least 0.15. ◁

In general, the spectral gap satisfies $\gamma = 1 - |\lambda_2(\mathbf{G})| \geq 1 - \alpha$. As discussed in Section 2.8, a larger gap corresponds to faster asymptotic convergence.

Moreover, $|\lambda_2(\mathbf{G})|^t \leq \alpha^t$: on the same graph, reducing the damping factor α makes α^t decay more quickly, which favours faster convergence.

3.8 Choosing the Damping Factor

In the Google matrix

$$\mathbf{G} = \alpha\mathbf{S} + (1-\alpha)\frac{1}{n}\mathbf{e}\mathbf{e}^T,$$

the conventional choice of damping factor $\alpha = 0.85$, originally introduced by Brin and Page themselves [BP, p. 109], represents a compromise: for values of α close to 1, the surfer almost always follows links, and the computation may require more iterations; for lower values, teleportation is more frequent and convergence is faster, but the distribution of PageRank scores becomes flatter.

To assess the effect of α on the number of iterations, we compare the number of steps needed for the factor α^t to fall below the same threshold, for example $\varepsilon = 10^{-8}$.

Solving $\alpha^t \leq \varepsilon$ for t gives

$$t \geq \frac{\log(10^{-8})}{\log \alpha}.$$

With $\alpha = 0.85$, 114 steps suffice, whereas $\alpha = 0.99$ requires 1,833: over sixteen times as many. The effect of α on the number of iterations is examined further in Section 5.3.2.

In the random-surfer model, this value describes a surfer who, at each step, follows the transitions described by $\mathbf{S}$ with probability 0.85 and jumps to a random Web page with probability 0.15.

Chapter 4

Computational Algorithms

The Google matrix $\mathbf{G}$ is dense: storing it in full would require n^2 entries. Even a graph with $n = 10^6$ pages would require 10^{12} elements. By exploiting the sparse structure of the links, however, the PageRank vector can be computed without explicitly constructing $\mathbf{G}$.

To this end, we present the power method (or *power iteration*), which iteratively updates the score vector, and a Monte Carlo method, which estimates the vector through independent random walks. For both methods, we discuss the computational procedure, the computational cost and the control of accuracy.

4.1 Power Iteration

We choose an initial distribution $\bar{r}(0)$, which we take hereafter to be the uniform distribution $(1/n, \ldots, 1/n)$, and successively compute

$$\begin{aligned} \bar{r}(1) &= \bar{r}(0)\mathbf{G}, \\ \bar{r}(2) &= \bar{r}(1)\mathbf{G}, \\ &\vdots \end{aligned}$$

In general,

$$\bar{r}(t+1) = \bar{r}(t)\mathbf{G}. \tag{4.1}$$

By (3.6), the sequence converges to the PageRank vector. The algorithm performs one vector-matrix product at each step, without computing the powers of the matrix $\mathbf{G}$.

Since $\mathbf{G}$ is stochastic, the vector $\bar{r}(t)$ remains a probability distribution at every step, so no further normalisation is needed.

Example 4.1. Consider again the graph in Figure 3.1, adding teleportation with $\alpha = 0.85$ as discussed in Section 3.8.
From (3.4) we obtain

$$\mathbf{G} = \begin{pmatrix} 0.475 & 0.475 & 0.05 \\ 0.475 & 0.05 & 0.475 \\ 0.05 & 0.90 & 0.05 \end{pmatrix}.$$

Table 4.1. Evolution of the PageRank scores for the graph in Figure 3.1, with $\alpha = 0.85$.

t	$r_A(t)$	$r_B(t)$	$r_C(t)$
0	0.333333	0.333333	0.333333
1	0.333333	0.475000	0.191667
2	0.393542	0.354583	0.251875
5	0.375838	0.413949	0.210214
10	0.382605	0.396471	0.220924
20	0.381739	0.398740	0.219522
∞	0.381718	0.398795	0.219488

For example, $S_{A,A} = \frac{1}{2} \implies G_{A,A} = \alpha S_{A,A} + \frac{1-\alpha}{n} = 0.85 \cdot \frac{1}{2} + \frac{0.15}{3} = 0.475$.

Starting from $\bar{r}(0) = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$, we obtain the values in Table 4.1, rounded to six decimal places. The last row gives the limit of the sequence,

$$\bar{r} = \left(\frac{760}{1991}, \frac{794}{1991}, \frac{437}{1991} \right).$$

The scores oscillate around their limiting values, with progressively smaller errors. In this example, the second eigenvalue λ_2 of $\mathbf{G}$ is approximately -0.688: the error component associated with this eigenvalue changes sign at each step. Without teleportation, the stationary distribution of $\mathbf{P}$ was $(\frac{2}{5}, \frac{2}{5}, \frac{1}{5})$. That of $\mathbf{G}$ differs: with teleportation, B overtakes A. ◁

Storing $\mathbf{G}$ explicitly requires n^2 values. Moreover, each update computes n scores, each as a sum of n products: the cost per iteration is therefore $\Theta(n^2)$. These costs can be reduced by exploiting the sparsity of $\mathbf{P}$, which contains one nonzero entry for each link in the graph, for a total of $m = |E|$.

4.2 Sparse Computation

We define the column indicator vector $\mathbf{a} \in \{0,1\}^n$ of the dangling nodes, with

$$a_i = \begin{cases} 1 & \text{if } d_i^+ = 0, \\ 0 & \text{if } d_i^+ > 0. \end{cases}$$

The stochastic matrix $\mathbf{S}$, introduced in Section 3.5, can then be written as

$$\mathbf{S} = \mathbf{P} + \frac{1}{n}\mathbf{a}\mathbf{e}^T.$$

The added term replaces each zero row of $\mathbf{P}$ with the uniform distribution and leaves the other rows unchanged.

Example 4.2. For the graph in Figure 3.2, ordering the nodes as $A, \ldots, G$ gives

$$\mathbf{a} = (0, 0, 0, 1, 0, 0, 0)^T.$$

The correction changes only the row corresponding to D:

$$\mathbf{P} = \begin{pmatrix} 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \xrightarrow{+\frac{1}{7}\mathbf{a}\mathbf{e}^T} \mathbf{S} = \begin{pmatrix} 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

◁

Substituting into equation (3.5) gives

$$\mathbf{G} = \alpha\mathbf{P} + \frac{\alpha}{n}\mathbf{a}\mathbf{e}^T + \frac{1-\alpha}{n}\mathbf{e}\mathbf{e}^T.$$

From equation (4.1), using $\bar{r}(t)\mathbf{e} = 1$ since $\bar{r}(t)$ is a probability distribution, we have

$$\begin{aligned} \bar{r}(t+1) &= \bar{r}(t)\mathbf{G} \\ &= \bar{r}(t)\left(\alpha\mathbf{P} + \frac{\alpha}{n}\mathbf{a}\mathbf{e}^T + \frac{1-\alpha}{n}\mathbf{e}\mathbf{e}^T\right) \\ &= \alpha\bar{r}(t)\mathbf{P} + \frac{\alpha\bar{r}(t)\mathbf{a}}{n}\mathbf{e}^T + \frac{1-\alpha}{n}\mathbf{e}^T. \end{aligned} \tag{4.2}$$

- $\alpha\bar{r}(t)\mathbf{P}$ collects the contributions reaching each page through links, multiplied by α;
- $\alpha\bar{r}(t)\mathbf{a}\mathbf{e}^T/n$ evenly redistributes the total score of the dangling nodes, identified by the indicator vector $\mathbf{a}$, multiplied by α;
- $(1-\alpha)\mathbf{e}^T/n$ is the teleportation contribution, equal for every page.

In particular, the product $\bar{r}(t)\mathbf{P}$ is a vector with one value per page. Its jth component is the sum of the contributions reaching page j through links:

$$\left(\bar{r}(t)\mathbf{P}\right)_j = \sum_{i:\,(i,j)\in E} \frac{r_i(t)}{d_i^+}.$$

To compute this sum, the links are stored grouped by destination page: for each page j, we construct a *list* L_j containing the pairs (i, w_{ij}), with $(i,j) \in E$ and $w_{ij} = 1/d_i^+$. The weights are computed only once, before the iterations.

Example 4.3. The graph in Figure 3.2 yields the incoming lists:

j	L_j
A	$[(C, 1)]$
B	$[(A, \frac{1}{2})]$
C	$[(B, \frac{1}{2})]$
D	$[(A, \frac{1}{2})]$
E	$[(B, \frac{1}{2}), (G, 1)]$
F	$[(E, 1)]$
G	$[(F, 1)]$

The dangling node D receives a link from A, so L_D is not empty; however, D does not appear as a source in any list. Its score is redistributed through the uniform term. ◁

Equation (4.2) gives the following algorithm.

Algorithm 1 PageRank by power iteration

INPUT: Graph $D = (V, E)$ with $|V| \geq 1$, $0 < \alpha < 1$, tolerance $\tau > 0$, int $T_{\max} \geq 1$
OUTPUT: Score vector, number of iterations

$n \leftarrow |V|$; compute the out-degrees d_i^+
Construct the indicator vector $\mathbf{a}$ of the dangling nodes
For each $j \in V$, construct the list L_j of the pairs (i, w_{ij}) with $(i, j) \in E$ and $w_{ij} = 1/d_i^+$
$\bar{r}(0) \leftarrow (1/n, \ldots, 1/n)$; allocate a second score vector of length n
for $t = 0, \ldots, T_{\max} - 1$ **do**
    $b \leftarrow (1 - \alpha + \alpha \bar{r}(t)\mathbf{a})/n$
    for each $j \in V$ **do**
        $r_j(t+1) \leftarrow \alpha \sum_{(i,w_{ij}) \in L_j} r_i(t) w_{ij} + b$
    $\Delta_t \leftarrow \|\bar{r}(t+1) - \bar{r}(t)\|_1$
    if $\Delta_t < \tau$ **then**
        break
return $(\bar{r}(t+1), t+1)$

The uniform contribution is denoted by b and is recomputed at each iteration. The algorithm stops when $\Delta_t < \tau$, or after $T_{\max}$ iterations. The returned value $t+1$ is the number of iterations performed. Reaching $T_{\max}$ does not guarantee that the required tolerance has been met.

Initialisation requires $O(m+n)$ operations, where $n = |V|$ and $m = |E|$. Arithmetic operations are assigned unit cost, and list access and appending are assumed to take constant time and amortised constant time, respectively.

At each iteration, the scan visits the n nodes and the m edges, requiring $O(m+n)$ operations, while all the remaining operations together cost $O(n)$. The cost of one iteration is therefore $O(m+n)$. Writing T for the number of iterations performed, with $1 \le T \le T_{\max}$, the total cost is $O(T(m+n))$.

Memory is occupied by the n lists L_j, which contain m pairs in total, and by four vectors of length n:

- $\bar{r}(t)$: the current scores;
- $\bar{r}(t+1)$: the new scores;
- $\mathbf{a}$: the dangling-node indicator;
- $(d_i^+)_{i \in V}$: the out-degrees, used in the preparation.

The space required is therefore $O(m+n)$.

4.3 Stopping Criterion

Convergence to the PageRank vector is guaranteed, and Section 3.7 establishes its rate; it remains to decide when to stop the computation. In line 9, Algorithm 1 compares the vectors obtained in the last two iterations using the norm

$$\Delta_t = \|\bar{r}(t+1) - \bar{r}(t)\|_1 = \sum_{j \in V} |r_j(t+1) - r_j(t)|.$$

The PageRank vector is unknown, so the error cannot be computed directly; however, Δ_t provides an upper bound on it.

For two distributions $\bar{x}$ and $\bar{y}$, the teleportation term cancels and, since $\mathbf{S}$ is stochastic,

$$\|\bar{x}\mathbf{G} - \bar{y}\mathbf{G}\|_1 = \alpha\|(\bar{x} - \bar{y})\mathbf{S}\|_1 \le \alpha\|\bar{x} - \bar{y}\|_1.$$

Since $\bar{r}(t+1) = \bar{r}(t)\mathbf{G}$ by (4.1) and $\bar{r}\mathbf{G} = \bar{r}$ by (3.7), applying the inequality just obtained and the triangle inequality gives

$$\begin{aligned} \|\bar{r}(t+1) - \bar{r}\|_1 &\le \alpha\|\bar{r}(t) - \bar{r}\|_1 \\ &\le \alpha(\Delta_t + \|\bar{r}(t+1) - \bar{r}\|_1). \end{aligned}$$

Moving the term containing the error to the left-hand side,

$$\|\bar{r}(t+1) - \bar{r}\|_1 \le \frac{\alpha}{1-\alpha}\Delta_t.$$

To guarantee an error below ε when $\Delta_t < \tau$, it therefore suffices to choose $\tau = \frac{1-\alpha}{\alpha}\varepsilon$. This guarantee holds in exact arithmetic, whereas numerical computation also introduces rounding errors. Iterating the contraction inequality gives

$$\|\bar{r}(t) - \bar{r}\|_1 \le \alpha^t \|\bar{r}(0) - \bar{r}\|_1.$$

The error after t iterations is therefore at most α^t times the initial error.

4.4 Monte Carlo Method

PageRank is estimated by simulating N independent random walks. Each walk starts at a page chosen uniformly at random. Before each move, the walk terminates with probability $1 - \alpha$; otherwise, it continues according to the matrix $\mathbf{S}$. If c_j is the number of walks ending at page j, its PageRank r_j is estimated by

$$\widehat{r}_j = \frac{c_j}{N}.$$

Every walk ends at some page, so $\sum_{j \in V} c_j = N$, and thus $\sum_{j \in V} \widehat{r}_j = 1$. The estimator $\widehat{r}_j$ is unbiased [ALNO, Sec. 2]:

$$\mathbb{E}[\widehat{r}_j] = r_j, \qquad \mathrm{Var}(\widehat{r}_j) = \frac{r_j(1 - r_j)}{N}.$$

A walk takes exactly k steps if it continues k times and then stops, which occurs with probability $(1-\alpha)\alpha^k$. Let $\bar{u} = (1/n, \ldots, 1/n)$ be the initial distribution, L the number of moves and J the final page. By the law of total probability,

$$\Pr(J = j) = \sum_{k=0}^{\infty} (1 - \alpha)\alpha^k (\bar{u}\mathbf{S}^k)_j.$$

The distribution of the final node equals the PageRank vector [ALNO, Sec. 2]:

$$\bar{r} = (1 - \alpha) \sum_{k=0}^{\infty} \alpha^k \bar{u}\mathbf{S}^k.$$

To simulate the moves, the links are stored grouped by source page: for each page i, we construct an *array* O_i containing the destinations j with $(i, j) \in E$. A link is chosen by sampling an array index uniformly, at cost $O(1)$. If page i is dangling, a page is chosen uniformly among all the n available pages.

Example 4.4. The graph in Figure 3.2 yields the outgoing lists:

i	O_i
A	$[B, D]$
B	$[C, E]$
C	$[A]$
D	$[\,]$
E	$[F]$
F	$[G]$
G	$[E]$

The dangling node D has no outlinks, so O_D is empty; when a walk visits D, the next page is chosen uniformly among all pages. The weights $1/d_i^+$ need not be stored, since they equal the probability of sampling from O_i. ◁

The procedure described above gives the following algorithm.

Algorithm 2 PageRank by the Monte Carlo method

INPUT: Graph $D = (V, E)$ with $|V| \geq 1$, $0 < \alpha < 1$, int $N \geq 1$
OUTPUT: Estimate of the PageRank vector

```
Construct the arrays O_i of outgoing-link destinations
c_j ← 0 for every j ∈ V
for s = 1, ..., N do
    Sample i uniformly from V
    while a new uniform sample in [0, 1) is less than α do
        if O_i is empty then
            Sample i uniformly from V
        else
            i ← element sampled uniformly from O_i
    c_i ← c_i + 1
return (c_j/N)_{j∈V}
```

Initialisation requires $O(m+n)$ operations, where $n = |V|$ and $m = |E|$. Arithmetic operations are assigned unit cost, and random sampling and array access are assumed to take constant time.

The expected number of steps in a walk is $\mathbb{E}[L] = \alpha/(1-\alpha)$. The expected cost of N walks is therefore $O(N/(1-\alpha))$. Dividing the n counters by N takes $O(n)$. The total expected cost is

$$O\left(m + n + \frac{N}{1-\alpha}\right).$$

The memory required is $O(m+n)$: each of the m edges appears exactly once in the n arrays, and one counter is stored per node.

4.5 Comparison

Table 4.2 summarises the properties of the two methods. The number of iterations T and the number of walks N have different meanings: the costs alone do not indicate which method is faster for the same accuracy.

Table 4.2. Comparison of the algorithms for computing PageRank.

	Power iteration	**Monte Carlo**
Accuracy:	With a suitable stopping threshold, an error below ε is guaranteed when the threshold is met.	Increasing the number of simulated walks reduces the variability of the estimated scores.
Time:	$O(T(m+n))$	$O\left(m + n + \frac{N}{1-\alpha}\right)$ (expected)
Memory:	$O(m+n)$	$O(m+n)$

Chapter 5

Implementation

This chapter describes the GPU computation of PageRank by power iteration and the Monte Carlo method. For the former, several implementation choices based on the CuPy library are evaluated, with a CPU version as a baseline. Tests on graphs ranging from about 80 thousand to over 60 million nodes measure execution time, memory requirements and the accuracy of the results.

5.1 Execution Environment

All tests were run with the same Google Colab configuration, shown in Table 5.1. The CPU power iteration uses NumPy for vector operations and SciPy for sparse matrix operations. On the GPU, both methods are implemented with CuPy, whose interface is similar to that of the two libraries.

In power iteration, the sums over the lists L_j (Section 4.2) can be computed independently using the scores from the previous iteration. In the Monte Carlo method, several independent walks can be simulated at once. Both algorithms therefore lend themselves to parallel computation on a GPU.

Table 5.1. Execution environment used for the tests.

GPU	NVIDIA Tesla T4 (15.64 GB), driver 580.82.07
CPU	Intel Xeon at 2.0 GHz, 2 vCPUs
RAM	13.61 GB
Software	Python 3.13.15, NumPy 2.1.3, SciPy 1.16.3, CuPy 14.0.1, CUDA Runtime 13.0

5.2 Graphs and Data Structures

The tests used four directed graphs: *web-Google*, which represents web pages and hyperlinks, and *twitter_combined*, *GAP-twitter* and *Flickr*, which represent users and "follow" relationships on the respective social networks. Table 5.2 lists their characteristics.

The *web-Google* graph [SNAP-G] was used to compare power iteration on the CPU and the GPU and to study its convergence. On *twitter_combined*, the accuracy

Table 5.2. Characteristics of the graphs used.

Graph	Nodes	Edges	Dangling nodes
web-Google	875,713	5,105,039	136,259
twitter_combined	81,306	1,768,149	11,209
GAP-twitter	61,578,415	1,468,364,884	25,889,280
Flickr	1,487,058	17,034,807	243,193

of the Monte Carlo method was evaluated as the number of walks increases. *GAP-twitter* was used to assess the behaviour of both algorithms on a large graph. The *Flickr* dataset [MK] was used to evaluate a contact recommendation system. For *Flickr*, Table 5.2 reports the data of the graph as of 2 November 2006. Its nodes are the IDs appearing as endpoints of the edges observed by that date.

In the Twitter and Flickr graphs, edges point from the follower to the followed user, to whom the score is transferred. The *twitter_combined* and *Flickr* files already use this order [SNAP, SNAP-T, MK]; in *GAP-twitter* the two nodes appear in the opposite order and are swapped when the file is read [SS-T, SS-G]. For *GAP-twitter*, only the edge structure is used, ignoring the random weights included in the dataset.

For *twitter_combined*, 652,617 duplicate occurrences were removed, since in the model of Section 3.2 each edge occurs only once in the set E. This leaves 1,768,149 distinct edges, as reported in [SNAP-T].

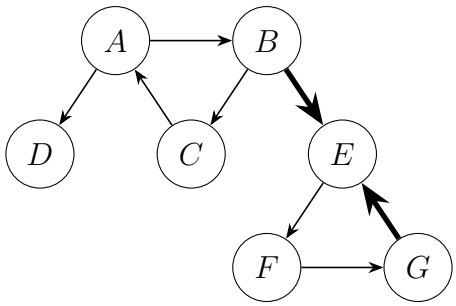
The graph is stored in Compressed Sparse Row format (hereafter CSR), which consists of three arrays:

- the column indices, stored one row after another;
- the corresponding values;
- the starting positions of the rows, with a final position marking the end of the array.

In power iteration, row j contains the pairs (i, w_{ij}) of the list L_j: the indices i and the weights w_{ij} are stored in the first two arrays. Example 5.1 follows.

In the Monte Carlo method, row i contains O_i: no weights are stored, since each outgoing link is chosen with probability $1/d_i^+$.

Example 5.1.



Row	A	B	C	D	$\boldsymbol{E}$		F	G
Source node	C	A	B	A	$\boldsymbol{B}$	$\boldsymbol{G}$	E	F
Weight	1	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\mathbf{\frac{1}{2}}$	**1**	1	1
Position	0	1	2	3	**4**	**5**	6	7
Row pointers	[0, 1, 2, 3, 4, 6, 7, 8]							

The lists of Example 4.3 are concatenated in the order $A, \ldots, G$, storing the source nodes and the weights in two separate arrays. Counting from zero, pointers 4 and 6 delimit L_E: positions 4 and 5 contain the pairs $(B, \frac{1}{2})$ and $(G, 1)$. The last pointer, 8, marks the end of the arrays. ◁

5.3 Power Iteration on CPUs and GPUs

The baseline CPU and GPU versions follow the same Algorithm 1. The code stores $\mathbf{P}^\mathsf{T}$ in CSR format, so that row j contains the incoming list L_j. The product $\bar{r}(t)\mathbf{P}$ is thus computed in transposed form as $\mathbf{P}^\mathsf{T}\bar{r}(t)^\mathsf{T}$.

The result is multiplied by α and the uniform contribution b is added, without constructing the matrix $\mathbf{G}$.

5.3.1 Execution Times

For the timing comparison, the iteration stops when $\Delta_t < 10^{-6}$. The baseline GPU version was modified in three steps:

1. Extracting and summing the scores of the dangling nodes is replaced by a dot product with a vector equal to 1 at the dangling nodes and 0 elsewhere.

2. The stopping criterion is checked every five iterations. This reduces how often Δ_t is computed, a value that the CPU must read from the GPU to decide whether to stop. Less frequent checks may entail a few additional iterations.

3. The matrix $\alpha\mathbf{P}^\mathsf{T}$, with weights α/d_i^+, is stored from the start. This avoids multiplying the result by α at every step.

Table 5.3. Comparison on web-Google: time to stopping, time per iteration, speedup over the CPU and maximum 1-norm deviation from the reference.

Version	Iterations	Time (ms) median [min; max]	Time per iteration (ms)	Speedup	Deviation ($\times 10^{-6}$)
Baseline CPU	62	1671.62 [1632; 1923]	26.96	1×	2.01
Baseline GPU	62	100.46 [99.74; 102.32]	1.62	16.6×	1.93
+ dot product	62	91.61 [90.77; 95.36]	1.48	18.2×	1.93
+ check every 5	65	45.59 [45.10; 48.00]	0.70	36.7×	1.24
+ α in weights	65	45.00 [44.37; 45.68]	0.69	37.1×	1.23

The comparison is run on *web-Google* with $\alpha = 0.85$, weights and scores in `float32` and the sum for Δ_t in `float64`; the deviation is measured against a reference in `float64` (threshold 10^{-12}). After a warm-up, each version was run seven times, in random order for the GPU versions, timing only the iterations up to stopping, checks included (Table 5.3). Times are measured with `time.perf_counter()`, synchronising the GPU before and after the computation.

On *web-Google*, the baseline GPU version achieves a speedup of 16.6× over the CPU; the modifications raise it to 37.1×. The deviation from the reference remains of the order of 10^{-6} in every version.

5.3.2 Convergence for Different Values of α

To verify the rate of convergence discussed in Section 3.7, power iteration was run on *web-Google* in double precision for several values of α, measuring the error $\|\bar{r}(t) - \bar{r}\|_1$. The error is measured against a reference PageRank computed in double precision. For each value of α, the reference is obtained with at least 300 iterations, continuing until $\alpha^t \leq 10^{-17}$. The curves stop when the error falls below 10^{-13}, to limit the influence of rounding.

The error decreases more slowly as α increases (Figure 5.1). For $\alpha = 0.85$, it falls below 10^{-8} in 93 iterations; the 114 iterations of Section 3.8 refer instead to α^t.

The reduction factor is the ratio of the error at one iteration to the error at the previous iteration. The values in Figure 5.2 are obtained from the slopes of the curves on a logarithmic scale. For $\alpha = 0.85$, the factor is about 0.842: about 84.2% of the previous error remains at each step in the range considered, within the theoretical maximum of 85% (Section 4.3).

Figure 5.1. Error $\|\bar{r}(t) - \bar{r}\|_1$ of power iteration on *web-Google* in double precision. The dashed lines show the bounds $\alpha^t \|\bar{r}(0) - \bar{r}\|_1$, obtained from the inequality in Section 4.3.

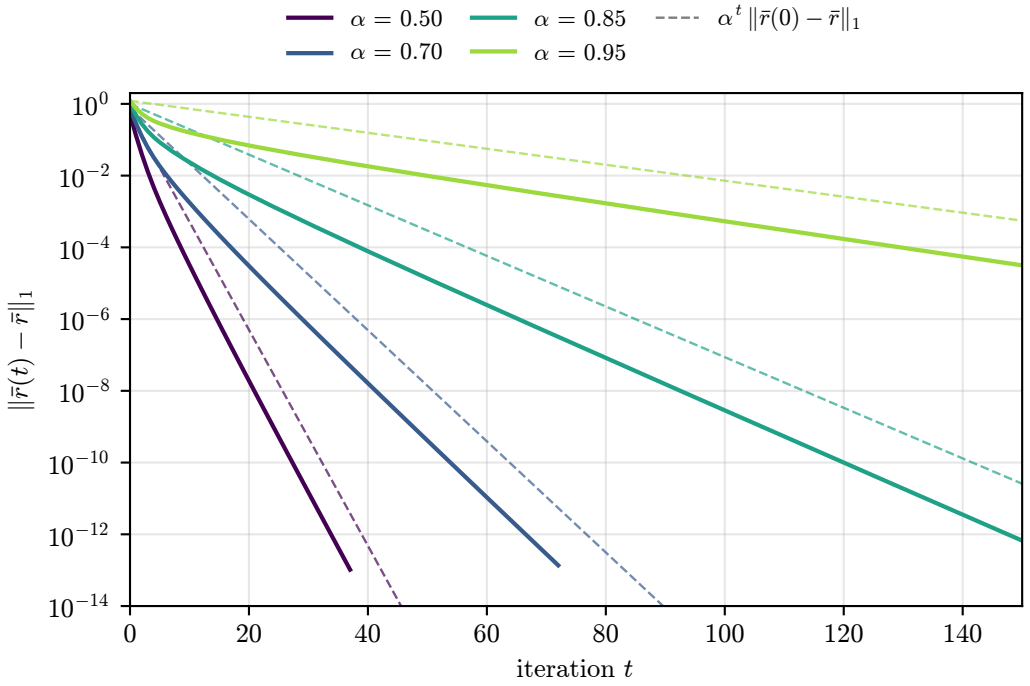
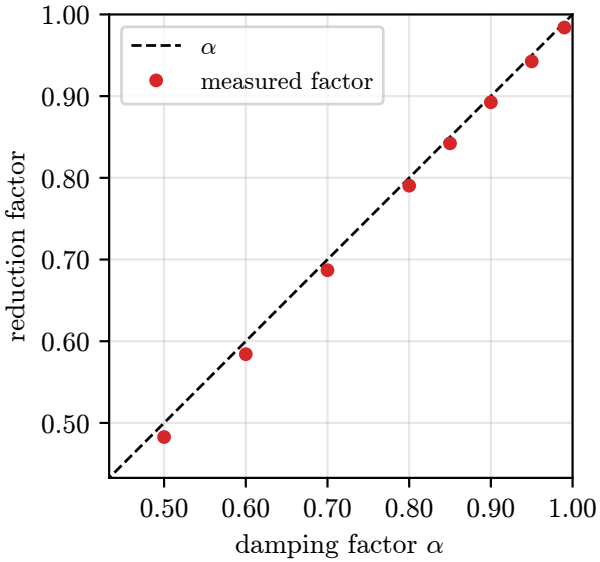


Figure 5.2. Error reduction factor per iteration for different values of α, estimated for $t \geq 10$ and errors above 10^{-11}. The dashed line represents α.



5.4 Monte Carlo on GPUs

The CuPy implementation processes batches of 10^6 walks to limit the memory required. The positions of the active walks are stored in a GPU array and updated in parallel. At the end of each batch, the occurrences of the final nodes are counted. The counts are summed across batches and divided by N to obtain $\widehat{r}$.

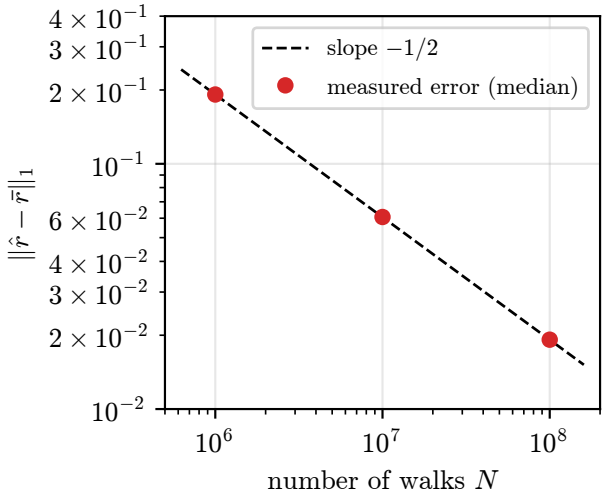
The implementation was validated on the graph in Figure 3.2, with $\alpha = 0.85$ and $N = 10^6$ walks:

- for every node, the deviation from the reference PageRank is less than two standard deviations, computed from the variance in Section 4.4;
- the error over the entire vector, measured in the 1-norm, is $2.17 \cdot 10^{-3}$;
- the mean number of steps per walk is 5.667, consistent with the theoretical value $\mathbb{E}[L] = \alpha/(1-\alpha) = 0.85/0.15 \approx 5.667$ from Section 4.4.

The tests on *twitter_combined* used $\alpha = 0.85$ and N equal to 10^6, 10^7 and 10^8 walks. Each test was repeated three times, with seeds 1, 2 and 3 for CuPy's `RandomState` generator. Median errors and times are reported. The reference PageRank was computed by power iteration in `float64`, reaching $\Delta_t < 10^{-12}$ after 142 iterations.

Increasing the number of walks from 10^6 to 10^8 reduces the error from about 0.192 to 0.0192 (Figure 5.3): a hundredfold increase in the number of walks reduces the error by about a factor of 10. The measured slope, -0.50, indicates a behaviour close to $1/\sqrt{N}$. The median times for 10^6, 10^7 and 10^8 walks are 0.145, 1.36 and 14.5 seconds, respectively, roughly proportional to N.

Figure 5.3. Error $\|\widehat{r} - \bar{r}\|_1$ of the Monte Carlo method on *twitter_combined* as a function of the number of walks N (median of three runs). The dashed line has slope $-1/2$.



5.5 Large-Scale PageRank

Both methods were run on *GAP-twitter*, distributed by the SuiteSparse Matrix Collection [SS-G], with about 61.6 million nodes and 1.47 billion edges. At this scale, data preparation and GPU memory use require particular care.

5.5.1 Data Preparation

The *GAP-twitter* file is read in two passes to build the row-pointer and index arrays described in Example 5.1. The index array is filled in RAM and saved to disk: it occupies about 5.87 GB, compared with the 11.75 GB needed to store all pairs of nodes as 32-bit integers.

After the conversion to CSR, the number of edges matches the official value and both sums of degrees, incoming and outgoing. Of the dangling nodes in Table 5.2, 19,926,185 are *isolated*, meaning that they also have no incoming edges, as reported by SuiteSparse [SS-G].

5.5.2 Power Iteration

In both Twitter graphs, all values stored in the matrix equal 1, while the division by d_i^+ is applied to the scores at each iteration. Dividing the score of node i by d_i^+ before the product is equivalent to multiplying it by the weight $1/d_i^+$ on each outgoing edge. We set $x_i = r_i(t)/d_i^+$ for $d_i^+ > 0$ and $x_i = 0$ otherwise; the mass of the dangling nodes is included separately in the uniform contribution b. Table 5.4 reports the results.

In the measurements of Table 5.4, moving from *twitter_combined* to *GAP-twitter* increases the time per iteration by about 800 times, against about 830 times as many edges. The number of iterations instead falls from 58 to 49: a larger graph does not necessarily require more iterations, consistent with the bound in Section 4.3, where the factor α^t does not depend on the size of the graph.

5.5.3 Monte Carlo Method

The implementation of Section 5.4 is used. Table 5.5 compares the two graphs with the same number of walks.

For the same number of walks, the time increases by about 30%, while the error increases about 23-fold. On *GAP-twitter*, with 10^9 walks and seed 1, the error falls to 0.135 in 177.6 seconds, and the set of the top ten nodes matches that obtained by power iteration.

5.5.4 Reducing Memory with a CUDA Kernel

The CSR representation used by CuPy requires a values array even when all values equal 1, as in Section 5.5.2. For a row with nonzero entries only in columns 2 and 5, it is enough to compute $x_2 + x_5$ instead of $1 \cdot x_2 + 1 \cdot x_5$, where x contains the scores already divided by the out-degree.

On *GAP-twitter*, the values array occupies 5.87 GB. To avoid this cost, a CUDA kernel, that is, a function executed in parallel on the GPU, was implemented to replace CuPy's matrix-vector product in power iteration without using the values array. The kernel is shown in Listing 5.1.

Table 5.4. Power iteration with $\alpha = 0.85$, scores in `float32` and stopping at $\Delta_t < 10^{-6}$. Times exclude loading. Median of ten runs for *twitter_combined*; a single run for *GAP-twitter*.

	twitter_combined	***GAP-twitter***
Iterations	58	49
Time (s)	0.0492	33.5
Time per iteration (ms)	0.849	685
Matrix memory (GB)	0.0145	11.99

Table 5.5. Monte Carlo with $\alpha = 0.85$ and $N = 10^8$. Times exclude loading. Medians of three runs for *twitter_combined*; a single run for *GAP-twitter*, with seed 1. References computed by power iteration: in `float64` with $\Delta_t < 10^{-12}$ for *twitter_combined*, and in `float32` with $\Delta_t < 10^{-6}$ for *GAP-twitter*.

	twitter_combined	***GAP-twitter***
Time (s)	14.5	18.8
Error in the 1-norm	0.0192	0.443

Listing 5.1. Matrix-vector product without the values array. Adapted from the CSR-vector scheme of [BG].

```
// 32 threads per row
int tid = blockIdx.x * blockDim.x + threadIdx.x;
int row = tid / 32;
int lane = tid % 32;
if (row >= n) return;

// Each thread sums part of the row
float sum = 0.0f;
for (int k = ptr[row] + lane; k < ptr[row+1]; k += 32)
    sum += x[col[k]];

// Sum of the partial results
for (int offset = 16; offset > 0; offset /= 2)
    sum += __shfl_down_sync(0xffffffff, sum, offset);
if (lane == 0) y[row] = sum;
```

The function runs with 256 threads per block, divided into eight *warps*, that is, groups of 32 threads, each assigned to one row. The arrays `ptr` and `col` contain the row pointers and the column indices; `x` contains the scores already divided by the out-degree, `y` the result, and `n` is the number of nodes. The `__shfl_down_sync` instruction combines the partial sums, and the thread with `lane == 0`, the first in the warp, writes the total. Removing the values array reduces the matrix memory from 11.99 to 6.12 GB. This variant reaches the same stopping threshold as CuPy in 49 iterations, with a 1-norm difference between the two vectors of about $7.8 \cdot 10^{-7}$.

In a dedicated comparison on *GAP-twitter*, with three runs per version, the median time of the kernel was 46.20 s, against 48.99 s for CuPy, an observed reduction of 5.7%. Even the slowest run of the kernel (46.36 s) was faster than the fastest run of CuPy (47.00 s). However, the session showed variable times, often longer than those in Table 5.4: the observed advantage requires confirmation through further measurements under stable conditions.

5.6 Contact Recommendation

So far, PageRank has been used to assess the global importance of nodes; we now introduce a personalised variant that assesses their importance with respect to a single user, in order to recommend new contacts on a social network.

5.6.1 Personalized PageRank

In *Personalized PageRank* (*PPR*), teleportation is not uniform but follows a distribution chosen for a user u [BP, p. 110]. The distribution is concentrated on node u: the random surfer returns to u both when teleporting and when reaching a dangling node.

The power iteration of Section 4.2 is modified by assigning the contribution nb to node u only, instead of adding b to every node. The scores favour nodes reachable from u through short, high-probability paths. The nodes with the highest scores are suggested, excluding u and the users that u already follows.

Teleportation concentrated on u does not guarantee that the matrix is strictly positive, so the argument of Section 3.6 does not apply. Since the dangling nodes return to u, however, the matrix remains stochastic, and teleportation, which is the same for every row, cancels in the difference: the inequality of Section 4.3 still holds, and the series of Section 4.4, modified in the same way, gives the unique PPR, to which power iteration converges with a factor of at most α.

Reciprocity is also taken into account: a user who follows u but is not followed by u is a potential contact to suggest. These users take priority and are ranked by their Personalized PageRank score. The remaining places are filled with the highest-ranked PPR candidates. Both strategies return ten suggestions per user.

Example 5.2. Suppose that three contacts are to be suggested to user u. Excluding u and the users that u already follows, Personalized PageRank ranks the candidates as A, B, C, D, E. Users C and E already follow u, but u does not follow them.

Personalized PageRank alone suggests A, B, C. PPR + reciprocity instead gives priority to C and E, in order of score, and fills the list with A: the suggestions are therefore C, E, A. ◁

5.6.2 Experimental Protocol

The experiments use the *Flickr* social network. Suggestions are computed on the graph of the links observed by 2 November 2006 and checked against the new links observed up to 3 December between users already present in the initial graph.

A random sample of 5,000 users with at least ten outgoing links in the initial graph is selected, without considering their subsequent activity, using the seed 20261004. For each user, ten suggestions are compared from global PageRank, Personalized PageRank alone and PPR with priority to reciprocity. The *Degree + reciprocity* method is also included: it ranks by in-degree the users who follow u without being followed back, and fills the list with the most-followed users. PageRank is computed in `float64`, with $\alpha = 0.85$ and stopping when $\Delta_t < 10^{-8}$.

- *HitRate@10* is the fraction of users for whom at least one of the ten suggestions matches a new link observed during the evaluation period.
- *Precision@10* is the fraction of the ten suggestions that match new links observed during the evaluation period, averaged over the 5,000 users.

Reciprocal links, whose reverse link already existed in the initial graph, are distinguished from non-reciprocal ones. All 95% confidence intervals are obtained by paired bootstrap resampling over users, with 10,000 resamples and seed 20261005, using the 2.5th and 97.5th percentiles of the resampled distributions.

5.6.3 Results

Table 5.6 reports the results of the recommendation methods. It shows that:

- Personalized PageRank identifies 491 new links and obtains at least one match for 348 users (HitRate@10: 6.96%), against 71 links and 61 users for global PageRank (HitRate@10: 1.22%);
- PPR + reciprocity identifies 795 links and obtains at least one match for 506 users (HitRate@10: 10.12%). Compared with PPR alone, Precision@10 rises by 0.61 percentage points [95% CI: 0.47; 0.75];
- Degree + reciprocity identifies 554 links and obtains at least one match for 353 users (HitRate@10: 7.06%). Compared with this method, PPR + reciprocity raises Precision@10 by 0.48 percentage points [95% CI: 0.40; 0.57].

The confidence intervals of both differences in Precision@10 are entirely positive, which supports the advantage of the combined method over both PPR alone and *Degree + reciprocity*.

Table 5.6. Recommendations for 5,000 Flickr users, with ten suggestions per user. Both metrics are computed over the entire sample. Square brackets give the 95% confidence intervals.

Method	**Links identified**	**HitRate@10** (%)	**Precision@10** (%)
Global PageRank	71	1.22 [0.92; 1.52]	0.142 [0.104; 0.184]
Personalized PageRank	491	6.96 [6.26; 7.70]	0.982 [0.866; 1.104]
Degree + reciprocity	554	7.06 [6.38; 7.78]	1.108 [0.966; 1.260]
PPR + reciprocity	795	10.12 [9.30; 10.96]	1.590 [1.426; 1.768]

The improvement over Personalized PageRank alone concerns the reciprocal links identified, which rise from 50 to 582, while the non-reciprocal ones fall from 441 to 213. In the sample considered, PPR + reciprocity therefore identifies more links overall, whereas Personalized PageRank alone obtains more matches among non-reciprocal links. The advantage of the combined method thus also depends on the type of links one wishes to favour. The evaluation concerns the links observed during the period considered and does not measure how much users appreciate the suggestions.

The system proves able to identify links observed later: on the metrics considered, the combination of Personalized PageRank and reciprocity is more effective than all the comparison methods evaluated, achieving higher values of both Precision@10 and HitRate@10.

5.7 Conclusions

The main findings of this work are summarised below.

The mathematical treatment of PageRank guided the GPU implementation of power iteration and the Monte Carlo method, which were tested on real networks of increasing size. The tests showed the slowdown of convergence as α increases and a power iteration error below the theoretical bounds of Section 4.3.

In the comparison between the two methods, power iteration is preferable for obtaining an accurate approximation of the entire PageRank vector. On *GAP-twitter*, the Monte Carlo method with 10^9 walks nevertheless identifies the same set of top ten nodes.

CuPy made it possible to implement both methods through library operations. On *web-Google*, the baseline GPU power iteration achieved a speedup of 16.6× over the baseline CPU version, rising to 37.1× with the implementation changes. Moving to larger graphs required particular care, above all in preparing the data and managing memory: an implementation must represent the model correctly and, at the same time, be executable with the available resources. On *GAP-twitter*, the library product was replaced by a CUDA kernel that exploits a specific property of the data, saving 5.87 GB of memory and, in a first comparison, achieving a shorter computation time than CuPy.

Power iteration was adapted to *Personalized PageRank* to build a contact recommendation system. On the *Flickr* social network, the system proved able to anticipate links observed later, identifying 795 such links and obtaining at least one match for 506 of the 5,000 users considered. Under the adopted protocol, PPR + reciprocity achieved the highest Precision@10 and HitRate@10 among the methods compared.

Possible future work includes evaluating the implementations on other GPUs and extending the recommendation experiments to other networks and time windows, in order to assess how much the results depend on the characteristics of the graph and on the observation period.

Bibliography

[ALNO] K. Avrachenkov, N. Litvak, D. Nemirovsky and N. Osipova, “Monte Carlo Methods in PageRank Computation: When One Iteration Is Sufficient”, *SIAM Journal on Numerical Analysis*, vol. 45, no. 2, pp. 890–904, 2007.

[BG] N. Bell and M. Garland, *Efficient Sparse Matrix-Vector Multiplication on CUDA*, NVIDIA Technical Report NVR-2008-004, 2008, Section 4.3, Figure 22.

[BP] S. Brin and L. Page, “The anatomy of a large-scale hypertextual Web search engine”, *Computer Networks and ISDN Systems*, vol. 30, nos. 1–7, pp. 107–117, 1998.

[EK] D. Easley and J. Kleinberg, *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*, Cambridge University Press, 2010, Chapter 14.

[LM] A. N. Langville and C. D. Meyer, “Deeper Inside PageRank”, *Internet Mathematics*, vol. 1, no. 3, pp. 335–380, 2004.

[LMB] A. N. Langville and C. D. Meyer, *Google’s PageRank and Beyond: The Science of Search Engine Rankings*, Princeton University Press, 2006.

[MK] A. Mislove, H. S. Koppula, K. P. Gummadi, P. Druschel and B. Bhattacharjee, “Growth of the Flickr Social Network”, *Proceedings of the 1st ACM SIGCOMM Workshop on Social Networks (WOSN’08)*, 2008. Dataset: `https://socialnetworks.mpi-sws.org/data-wosn2008.html`.

[MU] M. Mitzenmacher and E. Upfal, *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Analysis*, 2nd ed., Cambridge University Press, 2017.

[SNAP] Stanford Network Analysis Platform, `readme-Ego.txt`, `https://snap.stanford.edu/data/readme-Ego.txt`. Last accessed: September 2026.

[SNAP-G] Stanford Network Analysis Platform, *Google web graph*, `https://snap.stanford.edu/data/web-Google.html`. Last accessed: October 2026.

[SNAP-T] Stanford Network Analysis Platform, *Social circles: Twitter*, `https://snap.stanford.edu/data/ego-Twitter.html`. Last accessed: September 2026.

[SS-G] SuiteSparse Matrix Collection, *GAP/GAP-twitter*, `https://sparse.tamu.edu/GAP/GAP-twitter`. Last accessed: September 2026.

[SS-T] SuiteSparse Matrix Collection, *SNAP/twitter7*, `https://sparse.tamu.edu/SNAP/twitter7`. Last accessed: September 2026.